

XGBoost — Simple & Complete Notes

1. What is XGBoost?

XGBoost = eXtreme Gradient Boosting

It is an **optimized implementation of Gradient Boosting Decision Trees (GBDT)**.

The basic idea is:

Build decision trees **one after another**, where each new tree tries to correct the mistakes of the previous trees.

2. History

- Created by **Tianqi Chen** in **2014**.
 - Built on the idea of **Gradient Boosting**.
 - Became very popular after its success in the **Higgs Boson Kaggle competition**.
 - It is **open source** and supports multiple languages/platforms.
-

3. What can XGBoost do?

XGBoost can solve:

- **Classification** → predict a class
- **Regression** → predict a number
- **Ranking** → rank items/results

It also supports **custom loss functions and evaluation metrics**.

4. Why is XGBoost fast?

Parallel Processing

Boosting is sequential:

Tree 1 → Tree 2 → Tree 3 → Tree 4

But inside each tree, XGBoost can **parallelize the search for the best split across features**.

Feature 1 —

Feature 2 —|

Feature 3 —+→ Find best split

Feature 4 —

So training becomes faster.

Column Block Structure

XGBoost organizes data efficiently by **columns/features**.

This makes accessing feature values during split finding faster.

Cache Awareness

CPU cache is very fast memory.

XGBoost organizes data/computation to make better use of the CPU cache.

Result → faster computation.

Out-of-Core Computing

Suppose:

Dataset = 20 GB

RAM = 8 GB

The whole dataset cannot fit into RAM.

XGBoost can process the data in **chunks**:

20 GB

↓

Chunk 1 → process

Chunk 2 → process

Chunk 3 → process

...

Distributed Computing

XGBoost can distribute training across multiple machines.

Useful for **very large datasets**.

GPU Support

XGBoost can use a GPU to accelerate training.

Useful especially for computationally large workloads.

5. Why does XGBoost perform well?

Regularization

XGBoost adds a regularization penalty to the objective:

$$Objective = Loss + Regularization$$

So it doesn't only try to fit the training data.

It also tries to keep the model **less complex**.

→ Helps reduce **overfitting**.

Missing Values

XGBoost can automatically learn which direction a missing value should take during split finding.

So missing values can often be handled without manually filling every value first.

Efficient Split Finding

A tree needs to find good thresholds.

For example:

Age < 20

Age < 25

Age < 30

Age < 35

...

Checking every possible value can be expensive.

XGBoost uses techniques such as **weighted quantile sketching** to efficiently find useful candidate split points.

Tree Pruning

XGBoost controls unnecessary tree growth.

One important parameter is:

gamma

A split is made only when it provides sufficient improvement to the objective.

If improvement is too small:

No split

→ Helps control tree complexity.

6. XGBoost vs Gradient Boosting

Gradient Boosting	XGBoost
Sequential trees	Sequential trees
Corrects previous errors	Corrects previous errors
Basic implementation	Highly optimized
Less emphasis on system optimization	Cache/CPU/GPU optimization
Regularization depends on implementation	Built-in regularized objective
Can be slower	Generally highly optimized
Limited scalability	Distributed/out-of-core capabilities

 **Easy way to remember**

XGBoost = Gradient Boosting + Regularization + Speed + Scalability

7. Other Boosting Libraries

LightGBM

Developed by **Microsoft**.

Main focus:

Fast and memory-efficient gradient boosting

CatBoost

Main strength:

Handling categorical features effectively

★ Interview-ready definition

XGBoost is an optimized and regularized gradient boosting algorithm that builds decision trees sequentially to minimize prediction errors, while providing efficient training, missing-value handling, pruning, and scalability.

Most important things to remember

XGBoost →

Sequential trees → Correct errors → Regularization → Best splits → Fast training → Better generalization

XGBoost Regression — Mathematics Made Easy

The important part of this video is understanding **how XGBoost calculates each tree**. Let's make the mathematics simple.

1. Start with the average

Suppose we want to predict students' package based on CGPA.

Student CGPA Actual Package

A	6	3
B	7	4
C	8	6
D	9	8

First prediction = **mean of packages**.

$$\text{Mean} = \frac{3 + 4 + 6 + 8}{4} = 5.25$$

So initially XGBoost predicts:

Everyone → 5.25 LPA

2. Calculate residuals

Residual means:

$$\text{Residual} = \text{Actual} - \text{Predicted}$$

Actual Prediction Residual

3	5.25	-2.25
4	5.25	-1.25
6	5.25	+0.75
8	5.25	+2.75

The tree will now try to learn these **residuals**.

3. XGBoost's Similarity Score

This is the important difference from ordinary Decision Trees.

For XGBoost:

$$\text{Score} = \frac{(\sum \text{Residual})^2}{n + \lambda}$$

Where:

- $\sum Residual$ = sum of residuals
- n = number of samples in that node
- λ = regularization parameter

For now suppose:

$$\lambda = 0$$

Root node

Residuals:

$$-2.25, -1.25, 0.75, 2.75$$

Sum:

$$-2.25 - 1.25 + 0.75 + 2.75 = 0$$

Therefore:

$$Score = \frac{0^2}{4} = 0$$

So the **root score = 0**.

4. Try a split

Suppose we try:

$$CGPA < 7.5$$

Then:

Left

Residuals:

$$\begin{aligned} & -2.25, -1.25 \\ Score_L &= \frac{(-3.5)^2}{2} = \frac{12.25}{2} = 6.125 \end{aligned}$$

Right

Residuals:

$$\begin{aligned} & 0.75, 2.75 \\ Score_R &= \frac{(3.5)^2}{2} = \frac{12.25}{2} = 6.125 \end{aligned}$$

5. Calculate Gain

XGBoost compares the split using:

$$\boxed{Gain = Score_L + Score_R - Score_{Parent}}$$

Therefore:

$$Gain = 6.125 + 6.125 - 0$$

$$\boxed{Gain = 12.25}$$

A higher gain means a better split.

XGBoost tries different thresholds and chooses the split giving the best gain.

6. What is the tree actually learning?

This is the key idea:

The tree is not directly predicting package.

It is learning:

How much should the current prediction be corrected?

For example:

Initial prediction = 5.25

↓

Tree predicts correction

↓

Add correction

↓

New prediction

7. Learning Rate

Suppose a leaf gives a correction of:

2

and learning rate is:

$$\eta = 0.1$$

Then actual correction is:

$$0.1 \times 2 = 0.2$$

So:

$$NewPrediction = OldPrediction + 0.2$$

If old prediction was 5.25:

$$NewPrediction = 5.45$$

The learning rate prevents the model from making **very large corrections at once**.

8. Overall XGBoost Regression Process

Remember this flow:

Actual Y



Mean prediction



Calculate residuals



Calculate similarity scores



Try different splits



Calculate gain



Choose best split



Build tree



Calculate tree correction



× Learning Rate



Update prediction



Calculate new errors



Build next tree



Repeat

9. Why is XGBoost different from a normal Decision Tree?

A normal Decision Tree commonly uses:

- **Gini / Entropy** → classification
- **MSE** → regression

XGBoost uses an **objective function based on gradients/hessians**, and in the simplified regression derivation you are learning, this leads to the **similarity score and gain** calculations.

So don't memorize it as:

"XGBoost uses similarity score instead of everything else."

More accurately:

The similarity-score formula is part of the simplified/exact derivation of XGBoost's tree-building process.

★ Most important formulas

Residual

$$r_i = y_i - \hat{y}_i$$

Similarity score

$$Score = \frac{(\sum r_i)^2}{n + \lambda}$$

Gain

$$Gain = Score_L + Score_R - Score_P$$

Prediction update

$$F_m(x) = F_{m-1}(x) + \eta \cdot Tree_m(x)$$

Easy memory:

Mean → Residual → Score → Split → Gain → Tree → Learning Rate → New Prediction

That is the core mathematics you need from this XGBoost regression video.